Defining Success

How do we know if a software project succeeded?

The traditional answer is: **on time, on budget, with required features**. This sounds reasonable but has problems.

The Problem with Traditional Metrics

The Standish Group's CHAOS report famously claims that about 66% of software projects fail — they're over budget, late, or missing features. Only 30-35% are "successful."

But this framing assumes we knew exactly what to build from the start. In reality:

- Requirements change as stakeholders learn what they actually need
- The market shifts during development
- Early assumptions turn out to be wrong
- Better solutions emerge once work begins

If a team intentionally changes scope mid-project because they discovered a better approach, is that failure? Under traditional metrics, yes. In reality, it might be the smartest thing they could do.

Value Delivered, Not Plans Followed

A better definition of success: **Did we solve the problem? Did we deliver value?**

This shifts the focus from:

- Hitting arbitrary dates → Delivering working software
- Following the original plan → Responding to what we learn
- Measuring output (features shipped) → Measuring outcomes (problems solved)

What This Means in Practice

This doesn't mean schedules and budgets don't matter. They do. Constraints are real.

But it means:

- We plan iteratively, not all upfront
- We embrace learning and adaptation
- We measure success by whether users' problems get solved
- We treat changing requirements as normal, not as failure

The goal is software that works and solves real problems — not software that matches a document written before anyone understood the problem.